Accelerating ColBERT Multi-Vector Search with PDX/BOND

A Controlled Evaluation of Candidate Generation and Exact-Safe Dimension Pruning

Project Report

Author:	[**TODO: author name**]
University:	[**TODO: university**]
Programme:	[**TODO: programme / degree**]
Course:	Data Engineering [**TODO: official course code**]
Supervisor:	[**TODO: supervisor**]
Submission date:	[**TODO: submission date**]

All final numerical claims in this report are generated from the audited release artifacts under `results/final/` at clean Git commit `0cc6145`.

Abstract

ColBERT-style late-interaction retrieval represents queries and documents as sets of token vectors and scores each document with MaxSim, the sum over query tokens of the maximum token-level similarity. This report investigates whether the vertically decomposed PDX layout and BOND-style branch-and-bound dimension pruning can reduce the cost of exact ColBERT MaxSim search. All final claims are drawn from a controlled, single-machine, single-process benchmark with matched work budgets, interleaved repetitions, and audited release artifacts; earlier uncontrolled speedup figures were withdrawn after a measurement audit. Three findings emerge. First, indexing all document token vectors in a flat inverted-file (IVF) index and reranking retrieved candidates with a compiled exact MaxSim kernel yields a controllable quality–latency trade-off: on held-out SciFact queries, reranking 200 candidates per query recovers 97.5% of the exact top-10 while evaluating 3.9% of all query–document pairs. FAISS-IVF and PDX-IVF produce identical candidate pools and recovery curves, and neither engine shows a consistent latency advantage; this architecture is standard IVF practice, not a PDX-specific contribution. Second, and centrally, an independently implemented exact-safe BOND-MaxSim kernel preserves the exact top-10 on every held-out query but prunes too late to be useful: on raw 128-dimensional ColBERT embeddings it still evaluates 95.5% (SciFact) and 94.2% (NFCorpus) of the component products required by exhaustive scoring and runs roughly ten times slower than a precision-matched compiled exhaustive baseline measured in the same interleaved run. Third, an offline PCA rotation concentrates 90.9% of token energy in the first eight components and reduces evaluated products to 61.5–69.0%, yet both PCA arms remain about 8.7 times slower than their same-run exhaustive reference: dimension ordering improves the bound, but bound maintenance and irregular memory access dominate. The evidence indicates that exact-safe BOND dimension pruning is a poor fit for the tested ColBERT embeddings, while candidate generation plus exact reranking remains the practical architecture.

Contents

1 Introduction

1.1 Motivation

Late-interaction retrieval models such as ColBERT [3] achieve strong ranking quality by representing every query and every document as a *set* of contextualized token vectors rather than a single embedding. The price of this representational richness is paid at query time: the MaxSim scoring operator must compare every query token with every document token, so exhaustive scoring of a corpus grows with the product of query length, document length, corpus size, and embedding dimension. A large body of systems work, most prominently ColBERTv2 [7] and PLAID [6], addresses this cost with compression and approximate candidate pruning.

An orthogonal line of database research reduces the cost of high-dimensional similarity search by decomposing vectors *vertically*, that is, by dimension. BOND [1] scans dimensions incrementally and uses partial distances together with residual bounds to prune objects that can no longer enter the top-k. PDX [4] revisits this dimension-major layout for modern hardware and combines it with dimension-pruning search algorithms, including a PDX-BOND variant. These techniques were designed for single-vector nearest-neighbor search. Whether they transfer to the multi-vector MaxSim operator—where the scored object is a variable-length token matrix and the score is a sum of token-wise maxima—is an open engineering question, and it is the question this project set out to answer.

1.2 Research questions

The primary research question is:

> **RQ1.** Can BOND-style, exact-safe dimension pruning accelerate exact ColBERT MaxSim search over a strong compiled exhaustive implementation?

Three secondary questions structure the supporting experiments:

- **RQ2.** Why do the BOND residual bounds become useful, or remain loose, on normalized ColBERT token embeddings?
- **RQ3.** Can a flat token-vector index generate document candidates with high exact-MaxSim recall?
- **RQ4.** At matched rerank budgets, how do PDX-IVF and FAISS-IVF trade exact-ranking recovery for online latency?

The project explicitly does *not* claim inverted-file (IVF) candidate generation as a novel contribution. IVF is standard approximate nearest-neighbor practice; in this project it serves as a practical comparison architecture and as a fallback that was adopted while testing the BOND hypothesis.

1.3 Contributions

The report makes the following evidence-based contributions:

1. A packed, flattened token-index representation of variable-length ColBERT embeddings, with a token-to-document mapping that supports both vertically decomposed (PDX) and conventional (FAISS) indexes over the same data (section 3).
2. A controlled benchmark protocol with a single machine and process, matched work budgets, complete online timing boundaries, pinned threads, interleaved repetitions, and audited release artifacts (section 4). An internal measurement audit that led to the withdrawal of earlier, uncontrolled speedup claims is summarized as part of the methodology.

3. A controlled evaluation of IVF candidate generation followed by exact MaxSim reranking on SciFact and NFCorpus [8], showing identical FAISS-IVF and PDX-IVF recovery curves and no consistent engine-specific latency advantage (section 5).

4. An independently implemented, provably exact-safe BOND-MaxSim kernel and a matched-precision comparison against a compiled exhaustive MaxSim baseline. The central result is negative: on raw ColBERT dimensions the kernel evaluates 94–95% of the exhaustive component products and is roughly ten times slower than the exhaustive baseline (section 6).

5. A mechanism analysis using oracle pruning thresholds and an offline PCA rotation, showing that dimension ordering substantially improves pruning (down to 61.5% of component products) yet still does not overcome the kernel's bound-maintenance overhead (section 7).

Throughout the report, approximate IVF results and exact-safe BOND results are kept strictly separate, and no end-to-end retrieval-system speedup is claimed: all latency statements refer to a declared online query-processing boundary and to arms measured in the same interleaved run.

1.4 Report organization

Section 2 reviews late interaction, MaxSim, PDX, BOND, IVF, and related adaptive-distance work. Section 3 describes the data representation and the system components. Section 4 defines the controlled benchmark protocol. Sections 5 to 7 present the IVF, BOND, and PCA results. Section 8 interprets the evidence, section 9 discusses threats to validity, section 10 documents reproducibility measures, and section 11 concludes.

2 Background

2.1 ColBERT and late interaction

ColBERT [3] encodes queries and documents *independently* into sets of contextualized token vectors and delays their interaction until retrieval time. Let a query q be represented by token vectors $q_1, \ldots, q_m \in \mathbb{R}^D$ and a document d by token vectors $d_1, \ldots, d_n \in \mathbb{R}^D$. The late-interaction score is the MaxSim operator:

$$\text{score}(q, d) \;=\; \sum_{i=1}^{m} \max_{j=1,\ldots,n} \; q_i^\top d_j. \tag{1}$$

Each query token contributes its single best match among the document tokens, and these maxima are summed. Because token vectors are L2-normalized, the inner product in eq. (1) equals the cosine similarity. Exhaustive evaluation of eq. (1) over a corpus requires $m \times n \times D$ multiply–accumulate operations per document, which motivates both candidate pruning and cheaper scoring kernels.

ColBERTv2 [7] reduces the storage footprint of the token representation through residual compression, and PLAID [6] adds centroid-based candidate pruning inside a highly optimized retrieval engine. These systems establish that efficient multi-vector retrieval requires methods designed around the late-interaction operator itself rather than a single-vector distance alone.

An important structural property of eq. (1) is that the document score is *not* decomposable over individually retrieved tokens: knowing the best-matching document tokens for some query tokens does not determine the document score, because unretrieved query-token/document pairs still contribute their own maxima. This property drives the two-stage architecture analyzed in this report (section 3).

2.2 Vertical decomposition: BOND and PDX

BOND [1] stores vectors column-wise (dimension-major) and answers top-k nearest-neighbor queries by scanning dimensions incrementally. After processing a prefix of the dimensions, each candidate has a partial distance and a residual term that bounds its best possible final distance; candidates whose optimistic bound cannot enter the current top-k are pruned, so later dimensions are read for fewer and fewer objects. The effectiveness of this scheme depends on how much of the distance signal is concentrated in the early dimensions.

PDX [4] revisits the vertical layout for modern vector workloads. It decomposes vectors by dimension *within* blocks of the index, which preserves cache locality while enabling dimension-by-dimension kernels, and it pairs the layout with pruning-based search algorithms including a PDX-BOND variant that requires no offline transformation of the data. ADSampling [2] is related adaptive work in the approximate setting: it randomly rotates vectors and terminates distance computations early with probabilistic guarantees. In contrast, this project studies an *exact-safe* regime in which no correct top-k result may be lost, and asks whether such bounds remain selective when the scored object is a variable-length token matrix combined through eq. (1).

2.3 IVF candidate generation

An inverted-file (IVF) index clusters vectors into buckets (via k-means) and, at query time, scans only the `nprobe` buckets whose centroids are closest to the query vector. IVF is a standard, well-understood approximate nearest-neighbor technique; FAISS provides a reference implementation (`IndexIVFFlat`), and PDX provides an IVF variant (`IndexPDXBONDIVFFlat`) that applies its vertical layout and BOND-style scanning within probed buckets. In this project both engines index *individual document token vectors*, and their retrieved token hits are aggregated into document candidates that are subsequently reranked exactly (section 3).

2.4 Distance equivalence for normalized vectors

The PDX Python API exposes squared Euclidean distance only. For unit-normalized vectors x and y,

$$\|x - y\|_2^2 \;=\; 2 - 2\,x^\top y, \tag{2}$$

so nearest-neighbor ordering under squared L2 coincides with ordering by inner product or cosine similarity. Retrieved distances are converted back to similarities as $s = 1 - \|x - y\|_2^2/2$. All indexes in this study therefore operate on identical geometry despite exposing different metrics.

2.5 Benchmark datasets

SciFact and NFCorpus are drawn from BEIR [8], a heterogeneous zero-shot retrieval benchmark. SciFact is a scientific claim-verification corpus (5,183 documents in the configuration used here); NFCorpus is a nutrition-domain corpus (3,633 documents) with denser but still incomplete relevance labels. Embeddings are produced with PyLate [5] using the checkpoint `lightonai/GTE-ModernColBERT-v1`, which emits 128-dimensional L2-normalized token vectors.

3 Representation and System Architecture

3.1 Packed variable-length token representation

ColBERT produces a variable number of token vectors per document and per query. The project stores each collection in a packed format consisting of:

- one dense `values` matrix of shape (total tokens) $\times$ 128 containing all token vectors contiguously;
- one terminal `offsets` array of length (items + 1), so that item i owns the token rows $[\texttt{offsets}[i], \texttt{offsets}[i+1])$;
- parallel document and query identifier arrays.

This representation feeds every component in the study—the NumPy oracle, the compiled exact kernel, the BOND kernel, and both IVF engines—so all methods score exactly the same embeddings. On SciFact the corpus comprises 5,183 documents and 1,193,945 document token vectors; on NFCorpus, 3,633 documents and 864,703 token vectors.

3.2 Flattened token index and token-to-document mapping

For candidate generation, all document token vectors are indexed *independently* as points in $\mathbb{R}^{128}$, discarding document boundaries at index level. Two metadata arrays map each token-vector identifier back to its owning document and its token position within that document. This flattened design allows any off-the-shelf vector index to serve multi-vector retrieval, at the cost of an explicit aggregation step at query time.

3.3 Token retrieval, document selection, and exact reranking

Online query processing distinguishes three conceptually different stages:

1. **Token retrieval.** For each of the m query tokens, the index returns its top-L nearest document-token vectors ($L = 100$ throughout the controlled experiments).
2. **Document selection.** Every retrieved token hit is mapped back to its document. For each (query token, document) pair, only the best retrieved similarity is retained; contributions of query tokens with no retrieved hit in a document are replaced by zero. Summing these values yields an *approximate selector score*, and the top-C documents by this score are selected for reranking. The union of all documents hit by at least one token defines the *candidate pool*, whose full size is also measured as an oracle selection upper bound at fixed L.
3. **Exact MaxSim reranking.** The compiled exact kernel evaluates eq. (1) for the selected documents only, and the final document top-k is produced from these exact scores.

The zero-filled aggregation in stage 2 is deliberately *not* used as a final scorer. Token-only aggregation is not an exact document scorer for two reasons. First, a query token whose true best match in a document was not among the retrieved top-L hits contributes zero instead of its true maximum, underestimating the document score by an unknown amount. Second, the retrieved top-L lists are themselves truncated per query token, so different documents have different subsets of their token similarities observed. Early experiments confirmed empirically that ranking by this aggregate disagrees with exact MaxSim rankings; it is retained only as a cheap selection heuristic whose loss is measured explicitly (section 5).

3.4 Candidate engines

The controlled release compares two engines over the same flattened token index contents:

- **FAISS-IVF**: `IndexIVFFlat` over L2-normalized vectors with squared-L2 metric (eq. (2));
- **PDX-IVF**: `IndexPDXBONDIVFFlat` from the PDX library at pinned commit `fdc62f2`.

Both engines use the same IVF training sample, the same number of buckets, and the same `nprobe`; both feed the same aggregation, selection, and compiled exact reranking code. Index construction is an offline cost and is reported separately from online latency.

3.5 Exact reference kernels

A readable NumPy implementation of eq. (1) serves as the correctness oracle throughout. The *performance* reference is an independently implemented pybind11 C++17 kernel over the packed float32 inputs which validates dtype, shape, contiguity, and terminal offsets; releases the Python GIL; parallelizes query–document pairs with OpenMP; vectorizes the 128-dimensional dot products; and returns the full exact query–document score matrix. Its scores and deterministic rankings are tested against the NumPy oracle. A second, exact-safe BOND-MaxSim kernel is described together with its results in section 6.

4 Experimental Methodology

4.1 Measurement audit and withdrawn results

Before the controlled campaign, an internal audit examined the project's early wall-clock comparisons and found them unsuitable as performance evidence. The principal defects were: (i) PDX and FAISS candidate timers excluded token-to-document aggregation, candidate selection, reranking, and final ranking; (ii) PLAID was allowed up to 8,192 full scores per query while the IVF pipelines commonly reranked 50 documents, so the compared methods performed very different work; (iii) the exact baseline was a per-document Python/NumPy loop rather than a compiled kernel; and (iv) measurements from Windows and WSL environments appeared in the same figures. Additional issues included uncontrolled thread counts, no repeated or interleaved runs, parameter selection on the evaluation queries, favorable corpus prefixes, and IVF bucket counts that changed with corpus size in scaling plots. The historical headline ratios (approximately 30×, 39×, and 32×) were consequently withdrawn. Legacy artifacts are preserved under `results/legacy/` for mechanism and quality analysis only; no performance claim in this report relies on them.

4.2 Controlled benchmark protocol

All results in sections 5 to 7 were produced under the replacement protocol recorded in `docs/benchmark_protocol.md`, whose core requirements are:

- **One machine, one process.** All compared arms run in a single Python process on one machine (Ubuntu WSL2 on an AMD Ryzen 7 5800H), pinned to CPU affinity $\{0, 2, 4, 6\}$ (one hardware thread on each of four physical cores) with four OpenMP/BLAS threads.
- **Matched work and output contract.** Every arm must emit ranked document identifiers under the same exact MaxSim definition; approximate arms are compared at matched rerank budgets.
- **Complete online timing boundary.** The online timer starts with normalized query token vectors resident in memory and stops when final ranked document identifiers are available. It covers index search, token-to-document mapping, aggregation, candidate selection, exact reranking, and final top-k production. Query encoding and index construction are excluded and reported separately as offline costs. Reported ratios therefore describe this online boundary, not an end-to-end retrieval service.
- **Repetition schedule.** One untimed warm-up pass and five measured repetitions, with the arm order interleaved per repetition to reduce thermal and cache-order bias. Median and 95th-percentile (p95) latencies are reported and raw samples are preserved in the artifacts.

- **Provenance.** Every result JSON records the Git commit and dirty flag, input SHA-256 hashes, package versions, thread environment, CPU affinity, stage timings, and per-repetition samples. The release manifest accepts only artifacts from clean commit `0cc6145`.

4.3 Quality metrics

Two families of quality metrics are kept strictly separate:

Exact-ranking recovery. The primary architecture metric. For each query, the fraction of the *exact* MaxSim top-k documents that the approximate arm returns in its top-k (reported at $k = 10$ as exact recall@10). Candidate-pool recovery measures the same quantity against the entire retrieved pool, isolating pool-coverage loss from selector loss.

Qrels metrics. Standard relevance-judgment metrics (Recall@k, MRR@10) against the BEIR judgments. Because qrels are incomplete and the query samples are small, these metrics are insensitive to architecture differences (section 5) and are reported for completeness only.

4.4 Datasets, splits, and configuration

For each dataset the first 10 queries were used for pilot validation and selector-policy freezing, and the remaining **40 held-out queries** form the release comparison. On SciFact, the selector study found that a more complex `union_approx_and_count` policy changed exact recall@10 by at most ± 0.02 across budgets, so the simpler `approx_score` selector was frozen before the held-out runs. The candidate configuration is fixed at $L = 100$ token hits per query token and `nprobe` $= 8$; only the rerank budget $C \in \{50, 100, 200, 400, \text{full pool}\}$ varies (SciFact; NFCorpus uses $\{50, 200, \text{full pool}\}$). "Full pool" means that every unique document in the candidate pool is reranked.

4.5 Numerical precision contracts

Two distinct precision contracts are used, and their timings are never mixed: the **IVF comparison** reranks with the compiled float32 exact kernel, matching the float32 ColBERT embeddings, whereas the **BOND comparison** uses float64 products and accumulation in *both* the exhaustive and the BOND arm, because the exact-safe pruning proof requires controlled rounding (section 6). Consequently, float32 IVF latencies (e.g., the 2.93 s SciFact exhaustive median) must not be compared with float64 BOND-comparison latencies (e.g., the 4.27 s SciFact exhaustive median); each table names its own precision-matched baseline, and every BOND-to-exact ratio uses the exhaustive arm interleaved in the same run.

5 IVF Candidate-Generation Results

This section evaluates the approximate two-stage architecture of section 3.3: IVF token retrieval, document selection at budget C, and exact float32 MaxSim reranking. Results are approximate by construction and are evaluated against the exact ranking; they are reported separately from the exact-safe BOND results of section 6.

5.1 SciFact

Table 1 presents the held-out SciFact comparison and fig. 1 plots the resulting quality–latency frontier. Four observations follow directly from the artifacts.

Table 1: Controlled SciFact release comparison on 40 held-out queries (5,183 documents; $L = 100$, `nprobe` = 8; medians and p95 over five interleaved repetitions of the complete online boundary for all 40 queries). Exact R@10 is exact-ranking recovery against compiled exact MaxSim; the rerank ratio is the fraction of all query–document pairs evaluated exactly. Source: `scifact_ivf_test_40q_final.json`.

Arm	Median (s)	p95 (s)	Exact R@10	Mean C	Rerank ratio
Compiled exact	2.9333	3.1010	1.0000	5,183.0	1.0000
FAISS-IVF, C=50	0.3810	0.4003	0.8600	50.0	0.0096
PDX-IVF, C=50	0.3495	0.3575	0.8600	50.0	0.0096
FAISS-IVF, C=100	0.3814	0.4025	0.9325	100.0	0.0193
PDX-IVF, C=100	0.3973	0.4757	0.9325	100.0	0.0193
FAISS-IVF, C=200	0.5051	0.5885	0.9750	200.0	0.0386
PDX-IVF, C=200	0.5411	0.5541	0.9750	200.0	0.0386
FAISS-IVF, C=400	0.8483	0.9233	0.9900	390.9	0.0754
PDX-IVF, C=400	0.8820	0.9127	0.9900	390.9	0.0754
FAISS-IVF, full pool	1.2908	1.7934	1.0000	619.3	0.1195
PDX-IVF, full pool	1.2467	1.5615	1.0000	619.3	0.1195

Candidate pools are complete; residual loss is selector loss. For every one of the 40 held-out queries, the retrieved candidate pool (mean 619.3 documents) contains the entire exact top-10. Loss at finite C is therefore attributable to the zero-filled approximate selector, not to token retrieval. Reranking $C = 200$ documents per query recovers 0.975 of the exact top-10 while evaluating 3.86% of all query–document pairs; reranking the complete pool recovers the exact ranking with 11.95% of the pairs.

FAISS-IVF and PDX-IVF are quality-identical. The artifacts record that candidate pools, selected candidates, retrieved-hit counts, and final top-k rankings are equal between the engines at every budget; consequently their recovery curves in fig. 1 coincide exactly.

No consistent engine latency advantage. The latency winner changes with the budget (PDX is faster at $C = 50$ and full pool; FAISS is faster at $C = 100$, 200, and 400), and token-search stage medians differ by at most about 0.02 s across the SciFact arms. The evidence therefore does not support a PDX-specific retrieval advantage in this workload.

Qrels metrics cannot resolve the architecture differences. Qrels Recall@10 (0.985) and MRR@10 (0.902) are *identical* for every arm in table 1, even though exact-ranking recovery spans 0.86 to 1.00. With incomplete qrels and 40 queries, exact-ranking recovery is the sensitive metric; qrels metrics are reported for completeness only.

Figure 2 decomposes the online time by stage. At $C = 50$ token search dominates for both engines; at the full pool, exact reranking accounts for most of the latency. Offline index construction, excluded from these timings, took 18.52 s (FAISS) and 21.46 s (PDX) on SciFact.

5.2 NFCorpus transfer

The identical stack, online boundary, and configuration ($L = 100$, `nprobe` = 8, frozen selector) were applied to NFCorpus (3,633 documents; 864,703 token vectors; 40 held-out queries). Table 2 reports the results and the left panel of fig. 3 compares the recovery curves of both datasets.

The transfer is less forgiving than SciFact in one specific way: reranking the *entire* candidate pool reaches only 0.975 exact recall@10, and the pool contains the complete exact top-10 for 37 of the 40 queries. The residual 0.025 loss is therefore *candidate-pool coverage* loss—no selector operating on this pool can repair it—whereas on SciFact all loss at finite C was selector loss.

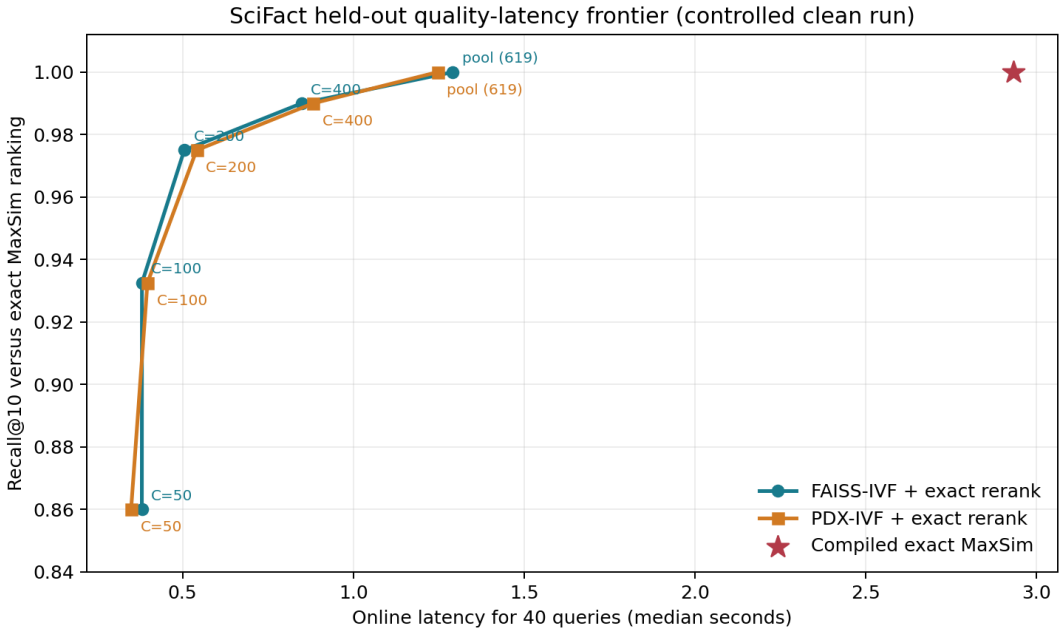

Figure 1: SciFact held-out quality–latency frontier from the controlled clean run (5,183 documents; 40 queries; $L = 100$, `nprobe` $= 8$; four pinned physical cores; median of five interleaved repetitions; online boundary only). Each point is one rerank budget C; the vertical axis is recall@10 against the exact MaxSim ranking. FAISS-IVF and PDX-IVF trace the same recovery curve, and both reach exact recovery when the full candidate pool (≈ 619 documents) is reranked, well below the compiled exhaustive latency (star).

Candidate pools and final rankings again agree between FAISS and PDX at every budget. The PDX token-search stage is slightly slower than FAISS in every matched NFCorpus arm (e.g., median 0.145 s versus 0.128 s at $C = 50$), but total online latency is nearly tied at $C = 200$ and at the full pool, so the earlier conclusion stands: the evidence shows no consistent PDX-specific advantage. Qrels Recall@10 moves only from 0.2150 (C=50) to 0.2191 (full pool, equal to the exact reference), confirming again that qrels cannot resolve these differences. Offline index construction took 12.27 s (FAISS) and 14.67 s (PDX).

6 Exact-Safe BOND-MaxSim: Design and Results

This section addresses RQ1 directly. Unlike the approximate IVF pipeline of section 5, the BOND-MaxSim kernel is *exact-safe*: it must return exactly the same top-k as exhaustive MaxSim,

Table 2: NFCorpus transfer comparison on 40 held-out queries (3,633 documents; same configuration and online boundary as table 1). Source: `nfcorpus_ivf_test_40q_final.json`.

Arm	Median (s)	p95 (s)	Exact R@10	Mean C	Rerank ratio
Compiled exact	1.4328	1.6469	1.0000	3,633.0	1.0000
FAISS-IVF, C=50	0.2114	0.2541	0.8275	50.0	0.0138
PDX-IVF, C=50	0.2365	0.2712	0.8275	50.0	0.0138
FAISS-IVF, C=200	0.3615	0.4032	0.9650	198.0	0.0545
PDX-IVF, C=200	0.3610	0.4265	0.9650	198.0	0.0545
FAISS-IVF, full pool	0.7019	0.7163	0.9750	388.1	0.1068
PDX-IVF, full pool	0.6952	0.7510	0.9750	388.1	0.1068

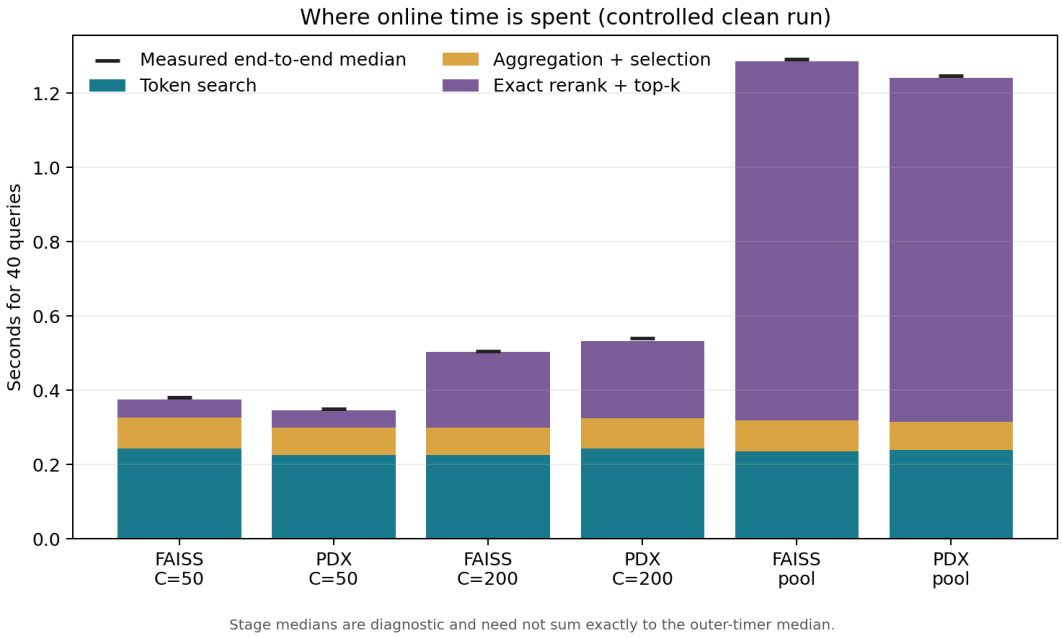

Figure 2: Online stage breakdown for the controlled SciFact run (40 held-out queries, five interleaved repetitions). Bars stack the median stage timers (token search; aggregation and selection; exact rerank and top-k) against the measured end-to-end median (black marker). At small budgets token search dominates; at the full candidate pool exact reranking dominates for both engines. Stage medians are diagnostic and need not sum exactly to the outer-timer median.

and any wall-clock result is rejected if the top-k set differs or an ordering difference exceeds the declared numerical tie tolerance. Both arms of every comparison in this section use float64 products and accumulation over the same float32 embeddings; per section 4.5, these timings are not comparable to the float32 IVF timings of section 5.

6.1 Kernel design

The project-local C++17 kernel stores each document in dimension-major (vertical) order and scans dimensions incrementally in fixed checkpoints $\{8, 16, 32, 64, 96, 128\}$. For query token q_i and document token d_j, after scanning the dimension prefix $[0, t)$, define the partial product and residual norms

$$p_{ij}(t) = \sum_{r<t} q_i[r]\, d_j[r], \qquad R_i^q(t) = \|q_i[t{:}]\|_2, \qquad R_j^d(t) = \|d_j[t{:}]\|_2. \tag{3}$$

The Cauchy–Schwarz inequality bounds each token similarity, $q_i^\top d_j \le p_{ij}(t) + R_i^q(t)\, R_j^d(t)$, which yields a document-level upper bound on the MaxSim score:

$$U_d(t) \;=\; \sum_i \max_j \Big(p_{ij}(t) + R_i^q(t)\, R_j^d(t) \Big) \;\ge\; \mathrm{score}(q, d). \tag{4}$$

Document residual norms are precomputed offline for every checkpoint; the inner maxima are initialized at $-\infty$ because all token similarities may be negative.

Pruning uses a conservative threshold: the kernel first scores a set of seed documents fully (the release configuration uses the first 500 documents in corpus order as deterministic prefix seeds), computes for each fully scored document a rounded-*down* lower bound on its exact score, and takes the kth largest of these lower bounds as the threshold, which can only increase as more documents complete. A live document is pruned only when its rounded-*up* upper bound eq. (4) is strictly below the threshold. Because (i) every threshold contribution understates a true score,

Table 3: Exact-safe BOND-MaxSim versus precision-matched compiled exhaustive MaxSim on 40 held-out queries (float64 products and accumulation in both arms; four pinned physical cores; five interleaved repetitions; one complete online timer). Every BOND arm returned the identical ordered top-10 for all 40 queries; the largest absolute score deviation across all arms is 1.78×10^{-14}. "Products" is the fraction of exhaustive component products actually evaluated. Latency ratios always use the exhaustive arm interleaved in the same run. Sources: `scifact_bond_raw_test_40q_final.json`, `scifact_bond_pca_test_40q_final.json`, `nfcorpus_bond_raw_test_40q_final.json`.

Arm	BOND med. (s)	Exact med. (s)	Ratio (×)	Pruned (%)	Products (%)
SciFact raw, prefix-500	46.4861	4.2740	10.88	22.44	95.47
SciFact raw, free oracle	48.1283	4.2740	11.26	36.32	92.43
SciFact PCA, prefix-500	35.5169	4.0591	8.75	86.30	69.02
SciFact PCA, free oracle	35.4903	4.0591	8.74	97.91	61.53
NFCorpus raw, prefix-500	20.3855	2.0397	9.99	23.36	94.16

(ii) the kth-largest lower bound cannot exceed the global kth-largest true score, and (iii) $U_d(t)$ overstates the unfinished document's true score, no document that belongs to the true top-k can be pruned; float64 accumulation with an explicit rounding allowance protects these inequalities near boundaries. Before any timing, the implementation was gated against a float64 NumPy oracle on randomized variable-length inputs, all-negative similarities, exact ties, forced-pruning workloads, and invalid inputs.

6.2 Held-out results on raw embeddings

Table 3 (first row) reports the primary decision measurement. On the 40 held-out SciFact queries the BOND kernel reproduces the exact ordered top-10 for every query, prunes 22.44% of the 207,320 query–document pairs, and yet still evaluates **95.47%** of the component products required by exhaustive scoring. Its median online latency is 46.49 s versus 4.27 s (p95: 48.11 s versus 4.41 s) for the precision-matched exhaustive arm in the same interleaved run—a **10.88-fold slowdown**. This is the main negative result of the project, and it is a controlled slowdown measurement, not a withdrawn-style estimate.

Why the bounds do not help: late pruning. The per-checkpoint pruning counts explain the discrepancy between the visible document-pruning percentage and the negligible arithmetic saving. Every single one of the 46,526 held-out SciFact prunes occurred at dimension checkpoint 96 of 128—none at checkpoints 8 through 64. A document pruned at dimension 96 has already paid 75% of its component products, so pruning 22.44% of documents saves only 4.53% of the arithmetic. Meanwhile the kernel pays for residual norms, per-pair upper bounds, threshold maintenance, branching, and partial-state writes on *every* surviving pair—costs the regular, SIMD-friendly exhaustive kernel does not incur. On raw normalized ColBERT embeddings the score signal is spread nearly uniformly across the 128 dimensions, so the Cauchy–Schwarz residual term stays large until almost the entire vector has been scanned. This answers RQ2 for the raw representation: the bounds become useful too late to offset their runtime overhead.

6.3 Transfer to NFCorpus

The frozen raw-order configuration was rerun unchanged on NFCorpus (table 3, last row; fig. 3, middle and right panels). The diagnosis transfers: the kernel preserves the exact ordered top-10 for all 40 held-out queries (maximum score deviation 7.11×10^{-15}), prunes 23.36% of pairs, evaluates 94.16% of component products, and is 9.99 times slower than its same-run exhaustive arm (20.39 s versus 2.04 s median). Pruning is again late, though marginally earlier than on SciFact: 730 documents prune at checkpoint 64 and 33,210 at checkpoint 96.

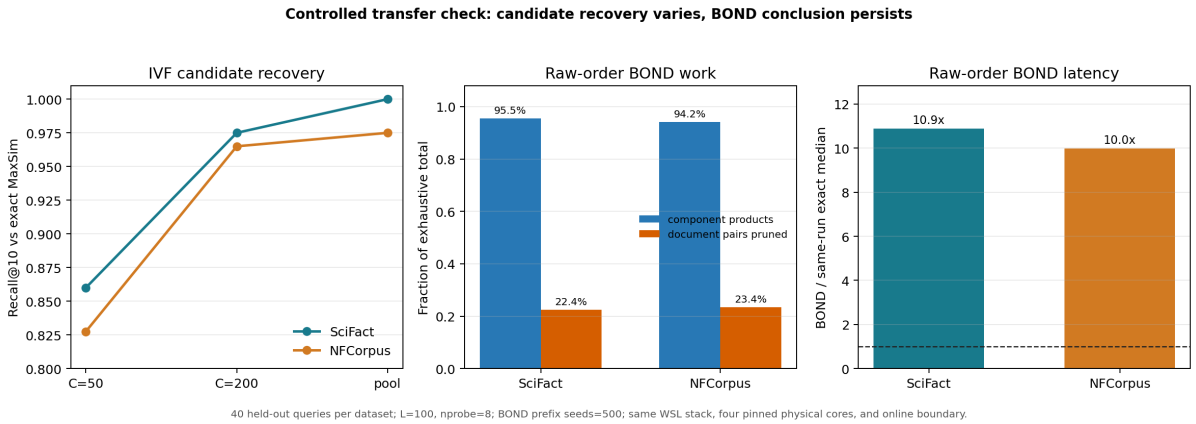

Figure 3: SciFact-to-NFCorpus transfer check (40 held-out queries per dataset; $L = 100$, `nprobe` $= 8$; BOND prefix seeds $= 500$; same WSL stack, four pinned physical cores, and online boundary). Left: IVF exact-ranking recovery versus rerank budget; NFCorpus saturates at 0.975 because its candidate pool misses part of the exact top-10 for 3 of 40 queries, whereas the SciFact pool is complete. Middle and right: raw-order exact-safe BOND evaluates 94–95% of exhaustive component products and remains about 10× slower than the same-run precision-matched exhaustive median on both datasets.

7 PCA and Dimension-Ordering Analysis

If late pruning is caused by energy being spread across dimensions, then a rotation that concentrates energy in the leading dimensions should make the BOND bound tighten earlier. This section tests that mechanism hypothesis with two diagnostic interventions. Both are explicitly *diagnostic*: the oracle seed policy is not deployable, and while PCA itself is a legitimate offline preprocessing step, here it is used to isolate the effect of dimension ordering on the bound.

7.1 Oracle seeds

The implementable threshold uses the first 500 corpus documents as deterministic seeds, so seed quality could in principle limit pruning. To remove this confounder, an oracle arm receives the exact top-10 documents as *free* seeds—their scoring cost is not charged—which yields the tightest threshold any seed policy could produce. On raw held-out SciFact embeddings (table 3, second row), oracle seeding raises document pruning from 22.44% to 36.32%, but the evaluated component products drop only from 95.47% to 92.43%, because every prune still occurs at checkpoint 96. The oracle arm is in fact marginally *slower* (48.13 s median, 11.26× the same-run exhaustive arm): earlier threshold tightening increases bound-check activity without saving meaningful arithmetic. Seed selection is therefore not the bottleneck; the bound geometry is.

7.2 Offline PCA rotation

An uncentered PCA is fitted offline on the document token vectors and the resulting orthogonal rotation is applied offline to both documents and queries. Because the rotation is orthogonal, it preserves inner products in exact arithmetic (measured float32 orthogonality error 2.53×10^{-8}), so the exact ranking is unchanged. The offline costs are small and reported separately: 0.513 s to fit, 0.230 s to transform, and 1.681 s to build the vertical index. After rotation, the first 8 of 128 components carry 90.91% of total token energy, and the first 64 carry 97.76%.

The effect on the mechanism is substantial (table 3, rows 3–4, and fig. 4). With the implementable prefix-500 seeds, document pruning rises to 86.30% and evaluated component products fall to 69.02%; with free oracle seeds, 97.91% of documents prune and only 61.53% of products are evaluated. Pruning also genuinely moves earlier: most prunes now occur at

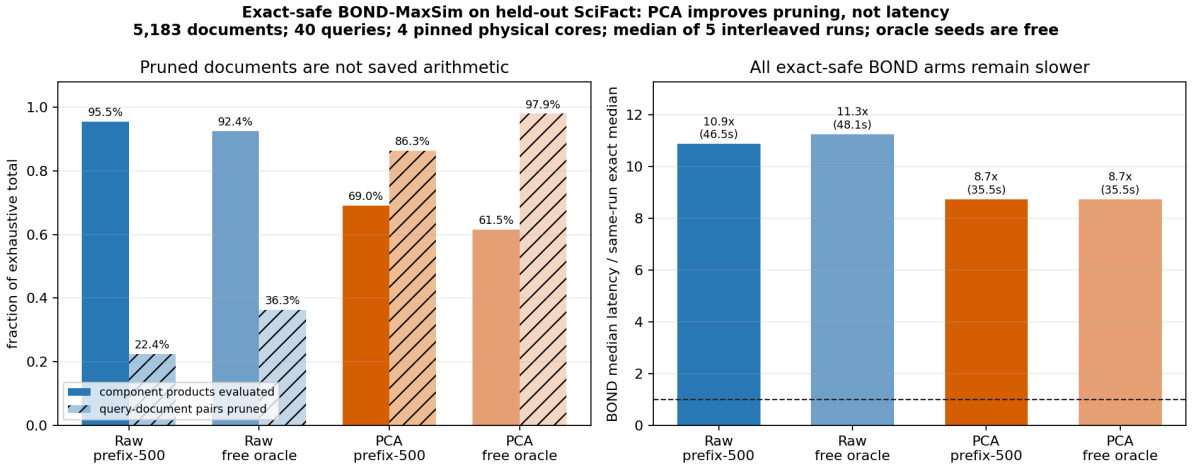

Figure 4: Exact-safe BOND-MaxSim on 40 held-out SciFact queries (5,183 documents; four pinned physical cores; median of five interleaved repetitions; oracle seeds are free and diagnostic). Left: fraction of exhaustive component products evaluated (solid) and query–document pairs pruned (hatched) for raw and PCA-rotated embeddings under prefix-500 and oracle seed policies. Right: BOND median latency relative to the precision-matched exhaustive median of the same run. PCA rotation improves pruning substantially (69.0% and 61.5% of products evaluated) yet every arm remains 8.7–11.3× slower than exhaustive scoring.

checkpoints 64 and 96 rather than exclusively at 96.

7.3 Arithmetic reduction does not become latency

Despite removing 31–38% of the component products, both PCA arms remain about 8.7× slower than the exhaustive arm interleaved in the same run (35.52 s and 35.49 s versus 4.06 s median). The latency gap narrows from 10.9× to 8.7× while the arithmetic drops by nearly a third, which localizes the remaining cost: bound maintenance, threshold updates, branching, partial-state reads and writes, and the irregular access pattern of surviving documents dominate the runtime of this kernel, not the multiply–accumulate work itself. Dimension ordering demonstrably helps the *bound*; it does not make this *kernel* competitive. This completes the answer to RQ2 and explains why inverse operation counts (as reported by the earlier NumPy instrumentation) must not be read as speedups.

8 Discussion

8.1 Answering the research questions

RQ1 (BOND acceleration): answered in the negative for this setting. Exact-safe BOND dimension pruning did not accelerate exact ColBERT MaxSim over the project's compiled exhaustive kernel. On raw 128-dimensional embeddings the kernel retained 95.47% (SciFact) and 94.16% (NFCorpus) of the exhaustive component products and ran 10.9× and 10.0× slower than the precision-matched exhaustive arm of the same interleaved run. The result is a controlled measurement with identical inputs, precision, thread budgets, and timing boundaries in both arms, and it replicates across two datasets.

RQ2 (bound behavior). The residual Cauchy–Schwarz bound stays loose because normalized ColBERT embeddings spread score energy almost uniformly across dimensions: on raw embeddings, every SciFact prune and almost every NFCorpus prune occurred at checkpoint 96 of 128, after 75% of the arithmetic was already spent. The PCA experiment isolates the two

remaining obstacles. Concentrating 90.91% of energy in eight components moves pruning earlier and removes 31–38% of products, so dimension order is genuinely part of the problem; but the persisting 8.7× slowdown shows that bound-maintenance and memory-irregularity costs, not multiply–accumulate volume, dominate this kernel. An exact-safe pruning kernel for MaxSim would need per-pair bookkeeping that is nearly free before it could profit from even a favorable dimension order.

RQ3 (flat token index as candidate generator): answered in the affirmative, with a dataset-dependent ceiling. With $L = 100$ and `nprobe` $= 8$, the flattened token index produced candidate pools containing the complete exact top-10 for all 40 SciFact queries and for 37 of 40 NFCorpus queries. Token-hit aggregation is not a valid final scorer (section 3.3), but as a selector feeding exact MaxSim reranking it supports a smooth, controllable quality–work curve.

RQ4 (PDX-IVF versus FAISS-IVF). The two engines produced identical candidate pools, identical exact-ranking recovery curves, and no consistent latency ordering across budgets and datasets; PDX token search was slightly slower in every matched NFCorpus arm while total latencies were nearly tied. The evidence indicates that, in this workload and at these corpus sizes, the vertically decomposed IVF implementation neither helps nor hurts candidate generation relative to a conventional IVF.

8.2 Interpretation and scope of claims

Three boundaries on interpretation deserve emphasis. First, the IVF pipeline is effective but *standard*: candidate generation followed by exact reranking is textbook approximate-retrieval architecture, and since FAISS reproduces the identical curve, none of that effectiveness is attributable to PDX. Second, no end-to-end speedup is claimed anywhere in this report. The online boundary excludes query encoding and index construction, and the IVF and BOND experiments use different precision contracts, so the only defensible latency statements are within-run, precision-matched ratios. Third, the measured negative BOND result is a statement about the tested exact-safe kernel on the tested embeddings—a document-oriented research prototype, not an upstream PDX block kernel—so it bounds this design point rather than the entire idea of dimension pruning for late interaction.

Within those boundaries, the practical guidance for multi-vector retrieval systems is clear: engineering effort is better spent on candidate generation quality (where NFCorpus shows a real coverage ceiling) and on fast exhaustive reranking kernels than on exact-safe dimension pruning of raw ColBERT representations.

9 Limitations and Threats to Validity

External validity. All release measurements come from one clean commit on one machine (Ubuntu WSL2, AMD Ryzen 7 5800H); hardware and operating-system generality is untested. SciFact (5,183 documents) and NFCorpus (3,633 documents) are small corpora, and results may change at larger scales or under different query- and document-length distributions. Only one candidate configuration ($L = 100$, `nprobe` $= 8$) appears in the held-out release; the quality–latency curves vary only the rerank budget C.

Baseline strength. The compiled exact kernel is validated and independently implemented, but it is a project baseline, not a claim to be the fastest possible MaxSim implementation. Similarly, the BOND-MaxSim kernel is an exact-safe, document-oriented research prototype rather than an upstream PDX block kernel; a substantially different engineering of the same

bound could shift the latency ratios, although the late-pruning geometry on raw embeddings would persist.

Statistical and evaluation limits. Each dataset contributes 40 held-out queries and five measured repetitions; interquartile ranges and p95 values are recorded, but formal significance testing was not performed. The qrels are incomplete, which is precisely why they proved insensitive (identical across all SciFact arms) and why exact-ranking recovery is the primary metric.

Residual tuning knowledge. IVF parameters originate from earlier exploration on these datasets. The validation/test query split prevents new selector tuning on evaluation queries, but it cannot erase all historical knowledge encoded in the chosen configuration.

Engine internals. PDX and FAISS may use different internal threading and memory layouts even under the same process-level thread limits; the stage timers bound, but do not eliminate, this confounder.

Missing comparison points. PLAID has not been rerun under a matched full-score budget, so no controlled PLAID comparison is reported. The historical PLAID numbers were withdrawn together with the other uncontrolled measurements (section 4.1).

Diagnostic arms are not methods. The oracle seed policy receives the exact top-10 for free and is a mechanism upper bound only; it is not deployable. PCA is an offline transformation whose fit, transform, and index-build costs are reported separately; the PCA arms demonstrate the effect of dimension ordering on the bound rather than an end-to-end method.

10 Reproducibility

The repository is organized so that every number in this report can be traced to an audited artifact:

- **Release artifacts.** The five JSON files under `results/final/` are the authoritative evidence. Each records the Git commit (`0cc6145`, `dirty=false`), SHA-256 hashes of the embedding and qrels inputs, package versions, CPU affinity and thread environment, warm-up and repetition counts, raw outer-timer samples, per-stage timings, exact-ranking recovery, qrels metrics, and, for BOND, per-checkpoint pruning counts and score-validation errors. `results/final/manifest.json` lists the accepted files with sizes and hashes. Historical results under `results/legacy/` and dirty-worktree pilots under `results/controlled/` are preserved for auditability but support no final performance claim.
- **Pinned environments.** Python dependency manifests are pinned; the PDX library is checked out at the audited commit `fdc62f2` (the controlled runner refuses a dirty or unexpected PDX checkout). Key package versions recorded in the artifacts include NumPy 2.4.6 and faiss-cpu 1.14.2 on Python 3.14.4.
- **Independent kernels and tests.** The compiled exact kernel (`cpp/exact_maxsim/`) and the exact-safe BOND kernel (`cpp/bond_maxsim/`) ship with build scripts and automated unit tests against the NumPy oracle, including tie, all-negative, forced-pruning, and invalid-input cases.
- **Controlled runners.** `experiments/pipeline/08` (IVF) and `experiments/pipeline/10` (BOND) implement the protocol of section 4.2; the repository `README.md` documents the exact `taskset` and environment-variable invocations used for the release runs.

- **Figures from artifacts.** Figures 1–4 are generated directly from the release JSON files by `experiments/pipeline/09`, `11`, and `12`, without hard-coded headline values.

Large embedding inputs remain outside Git; their SHA-256 hashes inside each artifact tie the recorded results to immutable inputs. The documentation set (`docs/benchmark_protocol.md`, `docs/audit_findings.md`, `docs/bond_kernel_design.md`, `docs/final_results.md`) records the protocol, the audit, the kernel safety argument, and the release summary, respectively.

11 Conclusion and Future Work

This project asked whether the PDX vertical layout and BOND-style exact-safe dimension pruning can accelerate ColBERT MaxSim search, and answered the question with a controlled, audited benchmark. Five conclusions are supported by the release artifacts:

1. **Flat token retrieval generates high-coverage candidates but is not a scorer.** Indexing individual token vectors and mapping hits back to documents yields candidate pools that contain the complete exact top-10 for all 40 held-out SciFact queries and 37 of 40 NFCorpus queries; token-hit aggregation itself, however, is not an exact document scorer and must be followed by exact MaxSim reranking.

2. **Candidate generation plus exact reranking gives a controllable quality–work curve—but it is standard IVF.** On SciFact, reranking 200 candidates per query recovers 97.5% of the exact top-10 with 3.9% of the query–document pairs, and full-pool reranking recovers the exact ranking with 12% of the pairs. This effectiveness is not a PDX contribution.

3. **FAISS-IVF and PDX-IVF are indistinguishable here.** Both engines produced identical candidate pools and recovery curves, and neither showed a consistent latency advantage across budgets and datasets.

4. **Exact-safe BOND pruning on raw ColBERT dimensions is a clear negative result.** The kernel preserves every exact top-10 but retains 94–95% of the exhaustive component products—all pruning occurs at dimension 96 of 128—and runs about 10× slower than the precision-matched compiled exhaustive baseline in the same interleaved run, on both SciFact and NFCorpus.

5. **Dimension ordering helps the bound, not the kernel.** An offline PCA rotation concentrating 90.9% of token energy in eight components cuts evaluated products to 61.5–69.0%, yet both PCA arms remain about 8.7× slower than their same-run exhaustive reference; bound maintenance and irregular memory access dominate.

Future work. Four directions follow naturally from the evidence: (i) a quality-matched PLAID comparison point under the same protocol and full-score budget; (ii) a larger corpus with a predeclared scaling protocol, to test whether the IVF coverage ceiling observed on NFCorpus widens or narrows with scale; (iii) a redesigned pruning kernel with near-zero per-pair bookkeeping (e.g., block-level rather than document-level bounds), which the PCA evidence suggests is the only path by which the improved pruning could become latency; and (iv) candidate-generation improvements (larger `nprobe` or L, or learned selection) targeted at the three NFCorpus queries whose exact top-10 never entered the pool.

References

[1] Arjen P. de Vries, Nikos Mamoulis, Niels Nes, and Martin L. Kersten. Efficient k-NN search on vertically decomposed data. In *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data*, 2002. doi: 10.1145/564691.564729.

[2] Jianyang Gao and Cheng Long. High-dimensional approximate nearest neighbor search: with reliable and efficient distance comparison operations, 2023. URL `https://arxiv.org/abs/2303.09855`.

[3] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020. URL `https://arxiv.org/abs/2004.12832`.

[4] Leonardo Kuffo, Elena Krippner, and Peter Boncz. PDX: A data layout for vector similarity search. *Proceedings of the ACM on Management of Data*, 2025. doi: 10.1145/3725333.

[5] LightOn. PyLate: Flexible training and retrieval for late interaction models. Software repository, version 1.5.0. URL `https://github.com/lightonai/pylate`.

[6] Keshav Santhanam, Omar Khattab, Christopher Potts, and Matei Zaharia. PLAID: An efficient engine for late interaction retrieval. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM)*, 2022. URL `https://arxiv.org/abs/2205.09707`.

[7] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2022. URL `https://arxiv.org/abs/2112.01488`.

[8] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Thirty-fifth Conference on Neural Information Processing Systems, Datasets and Benchmarks Track*, 2021. URL `https://arxiv.org/abs/2104.08663`.

A Implementation and Benchmark Details

A.1 Hardware and software environment

All release measurements were taken on a single machine: AMD Ryzen 7 5800H (16 logical CPUs), Ubuntu WSL2 (kernel 6.6.87.2-microsoft-standard-WSL2), Python 3.14.4 built with GCC 15.2.0. Processes were pinned with `taskset` to CPUs $\{0, 2, 4, 6\}$ (one hardware thread per physical core) with `OMP_NUM_THREADS=4`, `OMP_DYNAMIC=FALSE`, `OMP_PROC_BIND=TRUE`, `OMP_PLACES=threads`, and BLAS thread counts fixed at 4. Recorded package versions: NumPy 2.4.6, faiss-cpu 1.14.2, pdxearch 0.1 (PDX commit `fdc62f2d22b3793060abf633cb5407438c7f739b`, clean). Embeddings were produced with PyLate 1.5.0 and `lightonai/GTE-ModernColBERT-v1`.

A.2 Dataset statistics

Embedding dimension is 128 in both datasets. SciFact retains 49 qrels labels for the held-out queries and NFCorpus retains 1,250, which quantifies the difference in judgment density noted in section 4.3.

Table 4: Release dataset statistics (from the artifact `dataset` blocks). In both datasets, queries 0–9 were reserved for pilot validation and queries 10–49 form the held-out set.

Dataset	Documents	Doc. token vectors	Held-out queries	Query token vectors
SciFact	5,183	1,193,945	40	780
NFCorpus	3,633	864,703	40	461

A.3 IVF configuration

Both engines index all document token vectors with squared-L2 metric on L2-normalized inputs. SciFact: 2,186 buckets trained on 109,300 sampled points; NFCorpus: 1,860 buckets trained on 93,000 points; both with `nprobe` $= 8$, $L = 100$ hits per query token, seed 0, and the frozen `approx_score` selector. Offline build (train + load) times: SciFact 18.52 s (FAISS) and 21.46 s (PDX); NFCorpus 12.27 s (FAISS) and 14.67 s (PDX).

A.4 Online stage medians (SciFact)

Table 5: Median per-stage online times in seconds for 40 SciFact queries (five interleaved repetitions). Stage timers are diagnostic; they need not sum exactly to the outer end-to-end median.

Arm	Token search	Aggregation	Selection	Exact rerank + top-k
FAISS-IVF, C=50	0.2430	0.0730	0.0109	0.0491
PDX-IVF, C=50	0.2254	0.0651	0.0090	0.0461
FAISS-IVF, full pool	0.2351	0.0728	0.0109	0.9674
PDX-IVF, full pool	0.2398	0.0660	0.0091	0.9253

A.5 BOND kernel configuration

Dimension checkpoints $\{8, 16, 32, 64, 96, 128\}$; pruning threshold from the kth-largest conservative lower bound over fully scored documents; 500 deterministic prefix seeds (oracle arms receive the exact top-10 free); strict-inequality pruning with score tie tolerance 10^{-10}; float64 products and accumulation in both BOND and exhaustive arms. Offline vertical index construction: 1.755 s (SciFact raw), 1.681 s (SciFact PCA), 1.224 s (NFCorpus raw); estimated vertical index footprint on SciFact: 611 MB of values plus 57 MB of residual norms.

A.6 Per-checkpoint pruning counts

Table 6: Documents pruned at each dimension checkpoint (all held-out queries pooled). Raw-order pruning is concentrated at checkpoint 96; PCA moves most pruning to checkpoints 64 and 96.

Arm	dim 32	dim 64	dim 96	Total pruned	dim 128
SciFact raw, prefix-500	0	0	46,526	46,526	0
SciFact raw, oracle	0	0	75,299	75,299	0
SciFact PCA, prefix-500	9	83,722	95,177	178,908	0
SciFact PCA, oracle	40	121,166	81,782	202,988	0
NFCorpus raw, prefix-500	0	730	33,210	33,940	0

No pruning occurred at checkpoints 8 or 16 in any arm. Total held-out query–document pairs: 207,320 (SciFact), 145,320 (NFCorpus).

A.7 Numerical validation

BOND-versus-exhaustive score agreement on held-out queries: maximum absolute error 1.78×10^{-14} (SciFact raw), 1.07×10^{-14} (SciFact PCA), 7.11×10^{-15} (NFCorpus raw); ordered top-10 rankings identical in every arm with no non-tie inversions. The float32 compiled exact kernel used for IVF reranking matches the NumPy oracle with maximum absolute score error below 2×10^{-6} on the validation workloads, with identical rankings.